



Workshop::APIs and ReST

Luca Morandini

Head of AI Engineering – Melbourne eResearch Group

University of Melbourne

luca.morandini@unimelb.edu.au

Outline of the Workshop

- **Part 1:** Service-oriented Architectures
- **Part 2:** RPC (Remote Procedure Call) APIs
- **Part 3:** ReST (Representational State Transfer) APIs
- **Part 4:** The Mastodon ReSTful API

Part 1: Service-oriented Architectures

Disambiguation

- Software programs expose *interfaces* to enable interaction with other programs or users
- *User interfaces (UI)* are to communicate with users:
 - *Graphical User Interfaces (GUI)* are made with graphical elements and use pointers/touch input in addition to text
 - *Command-Line Interfaces (CLI)* are text-based interfaces where users type commands and get output
- *Application Programming Interfaces (API)* let programs interact with other programs within a single computer or across different computers via a network
- The way interfaces and programs are arranged in a system is called a *software architecture*

Service-oriented Architectures

- When an architecture is completely contained within the same machine, components can communicate directly, e.g. through *function calls*
- However, when components are distributed across compute nodes such a direct approach cannot be used (e.g. Assignment 2)
- Therefore, components have to interact in more loosely-coupled ways
- *Services* are often used to implement a distributed system by *combining* and *reusing* services to form a *Service-oriented Architecture (SoA)*

RPC vs ReST

- There are two main paradigms of service APIs:
 - *Remote Procedure Call (RPC)*: requests mimic local function calls, the *URL** *space* is generally shallow and not as important as in ReST since functions are identified in the payload. Examples are: *GraphQL*, *JSON-RPC*, *SOAP*
 - *Representational State Transfer (ReST)*: is built on the concept of *resources* that are identified by URLs (which tend to be deeply nested)

** it would be more precise to use the term URI (Uniform Resource Identifier) instead of URL (Uniform Resource Locator), since an URN (Uniform Resource Name) could be used in place of a URL, but we will go with the more common term here*

Part 2: RPC (Remote Procedure Call) APIs

SOAP Web-services Stack

- SOAP (*Simple Object Access Protocol*) web-services is a stack of protocols based on HTTP that covers every aspect of using a remote service, from service discovery, to service description, to the actual request/response
- SOAP and related protocols and languages are based on XML and precisely defined by XML schemas
- UDDI (*Universal Description, Discovery, and Integration*) is a protocol for describing, publishing, and locating web services
- WSDL (*Web Services Description Language*) is another protocol (more properly, a language) used to describe a single web service capabilities, endpoints, and methods
- SOAP is the protocol for actual requests to web-services

UDDI

Example of a UDDI response:

```
<?xml version="1.0" encoding="UTF-8"?>
<serviceList xmlns="urn:uddi-org:api_v2" generic="2.0" operator="UDDI_Node_Alpha">
  <serviceInfos>
    <serviceInfo serviceKey="uddi:example.com:service_helloWorld" *
      businessKey="uddi:example.com:business_acmeCorp">
      <name xml:lang="en">HelloWorldService</name>
    </serviceInfo>
  </serviceInfos>
</serviceList>
```

*URN

WSDL

Example of a WSDL response:

```
<definitions name = "HelloService"
  targetNamespace =
"http://www.examples.com/wsdl/HelloService.wsdl"
  xmlns = "http://schemas.xmlsoap.org/wsdl/"
  xmlns:soap = "http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns = "http://www.examples.com/wsdl/HelloService.wsdl"
  xmlns:xsd = "http://www.w3.org/2001/XMLSchema">
  <message name = "SayHelloRequest">
    <part name = "firstName" type = "xsd:string"/>
  </message>
  <message name = "SayHelloResponse">
    <part name = "greeting" type = "xsd:string"/>
  </message>
  <portType name = "Hello_PortType">
    <operation name = "sayHello">
      <input message = "tns:SayHelloRequest"/>
      <output message = "tns:SayHelloResponse"/>
    </operation>
  </portType>
```

...

```
<binding name = "Hello_Binding" type = "tns:Hello_PortType">
  <soap:binding style = "rpc"
    transport = "http://schemas.xmlsoap.org/soap/http"/>
  <operation name = "sayHello">
    <soap:operation soapAction = "sayHello"/>
    <input>
      <soap:body
        encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
        namespace="urn:examples:helloservice"
        use = "encoded"/>
      </input>
    <output>
      <soap:body
        encodingStyle=http://schemas.xmlsoap.org/soap/encoding/"
        namespace = "urn:examples:helloservice"
        use = "encoded"/>
      </output>
    </operation>
  </binding>
  <service name = "Hello_Service">
    <documentation>WSDL File for HelloService</documentation>
    <port binding = "tns:Hello_Binding" name = "Hello_Port">
      <soap:address
        location = "http://www.examples.com/SayHello/" />
    </port>
  </service>
</definitions>
```

SOAP

Example of a SOAP (POST) request:

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd">
  <soapenv:Header>
    <wsse:Security>
      <wsse:UsernameToken>
        <wsse:Username>Imorandini</wsse:Username>
        <wsse:Password>12345</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </soapenv:Header>
  <soapenv:Body>
    <m:GetUser xmlns:m="http://www.example.com/m">
      <m:UserId>rsinnott</m:UserId>
    </m:GetUser>
  </soapenv:Body>
</soapenv:Envelope>
```

JSON-RPC

- It closely resembles a function call, but in JSON

```
POST http://maths.com/arithmetic
```

```
{"jsonrpc": "2.0", "method": "add", "params": [2, 3], "id": 1}
```

```
=> {"jsonrpc": "2.0", "result": 5, "id": 1}
```

- `id` is required only to correlate requests with responses
- The POST method is generally used to send requests on HTTP, but JSON-RPC can run on WebSockets as well
- The URL space is shallow, since the function to call is defined in the `method` parameter
- Error messages do not necessarily return HTTP error codes (they return status 200 for most errors):

```
{"jsonrpc": "2.0", "error": {"code": -32601, "message": "Method not found"}, "data": {"method": "add"}, "id": 1}
```

GraphQL #1

- It commonly uses HTTP POST request (GET is supported, but less common)
- To retrieve data it uses a HTTP POST request with a JSON body, but with a string `query` parameter, that identifies:
 - The object to return (`user`)
 - The attributes that the object has to satisfy to be returned (`id: \"123\"`)
 - The attributes to return (`{ id name }`)

POST `http://mycompany.com/graphql`

```
{  
  "query": "{ user(id: \"rsinnott\") { id name } }"  
}
```

Response:

```
{"data":  
  {"user": {"id": "rsinnott", "name": "Richard O. Sinnott"}}  
}
```

GraphQL #2

- GraphQL is mainly used to retrieve/change data in a flexible way
- Functions can be defined and invoked:

POST <http://coderepository.com/graphql>

```
{  
  "query": "query GetRepo($org: String!, $repo: String!)  
    {organization(login: $org)  
      {repository(name: $repo)  
        {name description issues(states: OPEN, first: 5)  
          {nodes  
            {number title }  
          }  
        }  
      }  
    }",  
  "variables": {  
    "org": "acme",  
    "repo": "road-runner"  
  }  
}
```

GraphQL #3

- GraphQL can change data (*mutations*)
- It uses HTTP POST requests, the **query** parameter is a string, the attributes to return are listed after the query parameters

<http://mycompany.com/graphql>

```
{  
  "query": "mutation {  
    updateUser(id: \"ghopper\",  
      input: { name: \"Grace Hopper\" }) { id name } }"  
}
```

- Where **updateUser** is a mutation defined by the API

GraphQL #4

- GraphQL allows introspection

<http://mycompany.com/graphql>

```
{  
  "query": "{__schema {types {name}}}"  
}
```

Response:

```
{  
  "data": {  
    "__schema": {  
      "types": [  
        { "kind": "OBJECT", "name": "User" }  
      ]  
    }  
  }  
}
```


Part 3: ReST (Representational State Transfer) APIs

What is ReST?

"Representational State Transfer (ReST) is intended to evoke an image of how a well-designed Web application behaves: a network of web pages (a virtual state-machine), where the user progresses through an application by selecting links (state transitions), resulting in the next page (representing the next state of the application) being transferred to the user and rendered for their use."

- Roy Fielding

What's in a Name?



- Client requests Resource through its location (URL)
- Server sends representation of Resource
- This puts the client in a certain state
- Representation contains URLs allowing navigation
- Client follows URL to fetch another resource
- This transitions client into yet another state
- **Representational State Transfer**

RESTful applications *navigate* instead of *calling*

- Representations contain information about possible traversals
- Application navigates to the next resource depending on link semantics
- Navigation (links) can be delegated to the server

Resource-Oriented Architecture (RoA)

- A RoA is a system composed of a set of *resources* that are identified by URLs, manipulated by HTTP methods, represented by standard formats, and optionally connected via links
- A *resource* is anything that the system identifies with an URL and represents in a standard format
- If your users might want to:
 - create a hypertext link to it,
 - retrieve a representation of it,
 - annotate it,
 - change it,
 - ...then you should make it a resource

Resources and URL Examples

Resource	Potential URL
Version 1.0.3 of the software release	http://www.example.com/software/releases/1.0.3.tar.gz
The latest version of the software release	http://www.example.com/software/releases/latest.tar.gz
The first weblog entry for October 24, 2006	http://www.example.com/weblog/2006/10/24/0
A road map of Little Rock, Arkansas	http://www.example.com/map/roads/USA/AR/Little_Rock
Some information about jellyfish	http://www.example.com/wiki/Jellyfish
A list of the open bugs in the bug database	http://www.example.com/bugs/by-state/open
The relationship between two acquaintances, Alice and Bob	http://www.example.com/relationships/Alice;Bob

Mapping Actions to HTTP Methods

ACTION	HTTP METHOD
C reate Resource	PUT POST
R etrieve Resource	GET
U ppdate Resource	PUT
D efine Resource	DELETE

- PUT should be used when target resource URL is known by the client
- POST should be used when target resource URL is generated by the server
- Also: HEAD, OPTIONS, PATCH, CONNECT, TRACE

ReST Best Practices #1

- Use nouns, not verbs in URLs
- Make all HTTP GETs side-effect free. Doing so makes the request "safe"
- Use links in your responses to requests. Doing so connects your response with other data. It enables client applications to be "self-propelled", i.e. "what is the next step to take"

ReST Best Practices #2

- Make judicious use of query strings (use path segments for identity/hierarchy and use query parameters for selection/filters/paging)

✓ `/products/AXFC`

✗ `/products?product-id=AXFC`

✓ `/products?page=3`

✗ `/products/page/3`

- Make your URL space *discoverable*:

✓ `/buildings/ <= {"MC", "OldQuad"}`

✓ `/buildings/MC/rooms/ <= {"5210", "7201"}`

✓ `/buildings/MC/rooms/5210 <= {"Capacity": 10, "Floor": 5}`

✗ `/rooms <= {"MC":["5210", "7201"], "OldQuad": ["Main"]}`

✗ `/rooms/5210 <= {"Capacity": 10, "Floor": 5}`

✗ `/rooms-MC/ <= {"5210", "7201"}`

ReST Best Practices #3

- Use HTTP status codes:

SUCCESS

200 OK
201 Created
202 Accepted
etc

REDIRECTION

300 Multiple Choices
301 Moved Permanently
etc

CLIENT ERROR

400 Bad Request
401 Unauthorized
402 Payment Required
403 Forbidden
404 Not Found
405 Method Not Allowed
406 Not Acceptable
407 Proxy Auth Required
408 Request Timeout
409 Conflict
etc

SERVER ERROR

500 Internal Server Error
501 Not Implemented
502 Bad Gateway
503 Service Unavailable
504 Gateway Timeout
etc

ReSTful Services Should Have Uniform Interface

Uniform Interface :

- Identification of Resources: All important resources are identified by one (uniform) resource identifier mechanism (e.g. HTTP URL)
- Manipulation of Resources through standard representations: Each resource can have one or more representations. Such as [application/xml](#), [application/json](#), [text/html](#), etc. Clients and servers negotiate to select representation
- Self-descriptive messages: Requests and responses contain not only data but additional headers describing how the content should be handled. Such as if it should be cached, authentication requirements, etc. Access methods (actions) mean the same for all resources (universal semantics)

HTTP Methods

- HTTP methods can be
 - Safe
 - Idempotent
 - Neither
- Safe methods have no effect on the system state
- Idempotent methods can be called repeatedly but the effect of repeating a call is equivalent to making a single call
- GET, OPTIONS, HEAD – Safe
- PUT, DELETE – Idempotent
- POST – Neither safe nor idempotent

JSON-RPC vs GraphQL vs ReST

- While you can arguably use any API paradigm to implement an API, there are some general principles for choosing what most suits your specific use case
- ReST and GraphQL are focused on selecting and updating data, while JSON-RPC on executing remote procedures
- GraphQL can be more efficient than ReST, with focused queries, less navigation through URLs, and selection/mutation only of the fields of interest
- JSON-RPC is particularly suited for analytics and two-way communication between client and server

GraphQL vs ReST

- A ReST API to list students enrolled in a course may look like:

```
/courses/comp90024/students/  
=> ["5233456", "7865566", ...]  
/courses/comp90024/students/5233456  
=> {name: "Joe Doe", dateofbirth: "03-05-2004", ...}
```

- Computing the average age of all the students would require as many requests as there are students (plus the one request to list all their IDs)
- A GraphQL API may take only one request:

```
{  
  "query":  
    "query GetStudentBirthYears($courseId: ID!) {  
      course(id: $courseId) { id title  
        enrolledStudents { id firstName lastName yearOfBirth } } }",  
  "variables": {  
    "courseId": "comp90024"  
  }  
}
```

JSON-RPC vs ReST

- A ReST API to compute different derived properties of a social media post may look like:

```
/mastodon/posts/34254354575445456/sentiment
```

```
=> 0.8
```

```
/mastodon/posts/34254354575445456/topics
```

```
=> ["afl", "sport"]
```

- A corresponding JSON-RPC is more intuitive and flexible (it works on every text it is given)

```
/analysis
```

```
{"jsonrpc":"2.0","method":"sentiment","params":"Great Victory by the Geelong cats!","id":1}
```

```
=> {"jsonrpc":"2.0","result":0.8,"id":1}
```

```
/analysis
```

```
{"jsonrpc":"2.0","method":"topics","params":"Great Victory by the Geelong cats!","id":2}
```

```
=> {"jsonrpc":"2.0","result":["afl", "sport"],"id":2}
```

Part 4: The Mastodon ReSTful API

Mastodon ReSTful API #1

- To use the API, you have to create a Mastodon account (whose ID is contained in `MASTODON_ACCOUNT`) and then apply for an API key (in this example contained in the environment variable `MASTODON_TOKEN`)

- Get the account information:

```
curl -XGET  
"https://mastodon.au/api/v1/accounts/${MASTODON_ACCOUNT}"\  
  --header "Authorization: Bearer ${MASTODON_TOKEN}" -v
```

- List all the posts in the account:

```
curl -XGET  
"https://mastodon.au/api/v1/accounts/${MASTODON_ACCOUNT}/statuses"\  
  --header "Authorization: Bearer ${MASTODON_TOKEN}" -v
```


Mastodon ReSTful API #2

- Create a post on your account:

```
curl -XPOST "https://mastodon.au/api/v1/statuses"\
  --header "Authorization: Bearer ${MASTODON_TOKEN}"\
  --data-urlencode 'status=hello world' -v
```

- Get the ID from the response and put in an environmental variable named `POST_ID`

- List all the posts in the account (again) you should see the new post listed:

```
curl -XGET
"https://mastodon.au/api/v1/accounts/${MASTODON_ACCOUNT}/statuses"\
  --header "Authorization: Bearer ${MASTODON_TOKEN}" -v
```

Mastodon ReSTful API #3

- Delete the post:

```
curl -XDELETE "https://mastodon.au/api/v1/statuses/${POST_ID}"\  
  --header "Authorization: Bearer ${MASTODON_TOKEN}" -v
```

- Get the first 5 posts of the latest public posts from any user:

```
curl -XGET "https://mastodon.au/api/v1/timelines/public?limit=5"\  
  --header "Authorization: Bearer ${MASTODON_TOKEN}" -v
```

- Get the second page from the next portion of the [link](#) response header (from the second page, you can go to the first by taking the prev portion of the [link](#) response header)

References

- *Restful Web Services*, Leonard Richardson and Sam Ruby, 2007, O'Reilly.
- *Architectural Styles and the Design of Network-based Software Architectures*. Fielding, Roy Thomas, UC - Irvine, 2000.